

The Relative Smoothed Throughput Approach for Adaptive HTTP Streaming

Paulo Cezar Lacerda Neto
MídiaCom, Institute of Computing
Universidade Federal Fluminense
 Niterói, Brazil
 placerde@midiacon.uff.br

Célio V. N. Albuquerque
MídiaCom, Institute of Computing
Universidade Federal Fluminense
 Niterói, Brazil
 celio@midiacon.uff.br

Débora C. Muchaluat-Saade
MídiaCom, Institute of Computing
Universidade Federal Fluminense
 Niterói, Brazil
 debora@midiacon.uff.br

Abstract—HTTP streaming has become a practical technique to distribute multimedia content. This work proposes an adaptation method that improves the user's quality of experience when they consume video streaming over HTTP, compared to other methods considered state of the art. Based on the study of existing adaptation methods, we propose a new approach called Relative Smoothed Throughput (RST) method. Experiments with varying bandwidth conditions and multiple flows show that RST is able to outperform state of the art methods in efficiency.

Index Terms—Adaptive streaming, DASH, HAS

I. INTRODUCTION

It is now common practice for Internet video providers to use adaptive streaming over HTTP in order to distribute multimedia content to their users. This technique is useful in typical real world scenarios where there is variation among consumers' network conditions. For example, for the same content available on the Internet, there are usually users with a high-quality connection while others have little available bandwidth. In these cases if the video is offered at a fixed rate, it is not possible to provide the best experience according to each user network condition. In addition, during content playback, a same user may encounter variations in his/her network that may generate undesirable events, such as interruptions during video streaming or very frequent changes in video quality throughout playback, having a direct impact on user quality of experience.

In HTTP Adaptive Streaming (HAS) [4], media content is encoded in different qualities or representations, offering adequate rates for various types of user devices. This allows streaming to adapt to variations found in the network during playback, providing a better user experience. HAS is based on HTTP and therefore has the advantage of using a well-known technology already disseminated for web content distribution, different from specialized streaming protocols, which require the use of a specific media server. Another advantage of the HTTP protocol is that it generally does not face restrictions on firewalls operating in organizations and is commonly supported by CDNs (Content Distribution Networks) [15].

A variety of HAS implementations has been developed in recent times in the field of video streaming. Due to the variety of HAS implementations, a standard known as MPEG-DASH [14], or Dynamic Adaptive Streaming over HTTP, was

developed by the Motion Picture Experts Group (MPEG), to converge efforts in the development of this technology and has become a HAS reference specification. Jiang et al. [9] say that a robust video adaptation algorithm must aim at reaching fairness, efficiency and stability. The adaptation algorithm used by a HAS client (i.e., video player) decides the bitrate of the next segment to download and this is key to achieve these properties. This work focuses on the study of streaming over HTTP adaptation techniques and proposes a new approach capable of bringing improvements in the user quality of experience, through better efficiency, stability and fairness compared to state of the art methods.

This paper is organized as follows. In the next section, the HAS architecture is outlined, with its main components and operation mechanisms. In Section III, the two main classes of methods used for adaptive streaming over HTTP are presented. Then the method that served as the basis for the proposal presented in this paper is introduced. Section IV explains our adaptive streaming proposal. In Section V, we present ESFET, a tool we created to evaluate the performance of DASH adaptation algorithms. Section VI presents the results of the comparative evaluation of RST with other streaming methods. Finally, Section VII summarizes the strengths of our method and suggests future works.

II. HTTP ADAPTIVE STREAMING

The HAS architecture follows the client/server model. In a typical HAS architecture, the content is encoded and segmented on the server side and then stored in a storage component. When requested, the HAS server retrieves the content from the storage and transmits to clients using the HTTP protocol.

When there is a scenario with many concurrent accesses, a load balancer is usually used in front of one or more HAS servers, in order to distribute the load among them. Optionally, CDNs are used to distribute HAS content more efficiently for HAS clients.

Video segmentation is an important part of the content preparation. Before it becomes available for consumption, video content is divided into segments of specific duration, usually between 2 and 10 seconds for each representation. The video metadata, segment duration and available representations

are indicated to clients in a descriptor file. Once the content is prepared, including encoding and creation of the descriptor file, the player uses the HTTP protocol to request video segments and reproduce their contents. During video playback, the choice of each segment to be requested is performed by the player's adaptation algorithm.

III. RELATED WORK

There are several research papers that analyze and propose HTTP adaptive streaming techniques [3], [5], [7], [9], [10], [13]. Most of existing methods can be categorized according to their adaptation strategy in two types. The first category is throughput-based, because its methods use mainly the throughput measured in recent segment downloads to estimate the available bandwidth capacity and select the next segment to be downloaded. The second class of methods are buffer-based [7] and their main criteria to define the next segment's bitrate is based on the current length of the playout buffer.

The next two subsections present two state-of-the-art methods from these categories, with which we compare our proposal. Comparison results are presented in Section VI.

A. Smoothed HTTP Throughput Method

The work in [11] belongs to the throughput-based category and proposes a streaming over HTTP adaptation algorithm based on the measurement of the smoothed HTTP throughput (μ). This metric is calculated as the ratio of the segment duration or *msd* (media segment duration) to the last segment's download duration or *sft* (segment fetch time). We call this method Smoothed HTTP Throughput Method (ST), because it uses μ to choose the representation that will be requested from the server for each segment to be transferred. Its goal is to match the requested segment bitrate with the available bandwidth. This method increases the next segment bitrate when μ is greater than $1 + \epsilon$, being ϵ the switch-up factor, and reduces it when μ is smaller than γ , the switch-down threshold.

In order to avoid rebuffering, the bitrate's increase occurs only if the playout buffer content is larger than a minimum buffer level parameter. This method also adds a waiting time before transferring a new segment, avoiding buffer overflow if the available bandwidth is much higher than the video rate.

In order to decide the representation of the next segment, if μ is greater than $1 + \epsilon$ and the playout buffer is above the predefined minimum, the next segment representation will be one unit above the current one. It follows a step-wise approach to avoid sudden increases that could cause a rate reduction, when the network condition improvement is only momentary, generating quality instability.

The computation of ϵ is done based on Equation 1, where b_i corresponds to the bitrate of representation i and N is equal to the highest representation available for the video.

$$\epsilon = \max\left\{\frac{b_{i+1} - b_i}{b_i}, \forall i = [0, 1, \dots, N - 1]\right\} \quad (1)$$

To determine if it is necessary to choose a lower quality representation, it is verified if μ is less than γ , which represents

the threshold to reduce the bitrate. In this case, an aggressive bitrate decrease is performed and the algorithm selects the first representation with a bitrate less than $\mu * b_c$, where b_c is the current segment bitrate.

In addition to the bandwidth estimation dynamics, as previously mentioned, ST also limits the amount of segments transferred in advance to the playout buffer by defining an idle time between the download of two consecutive segments.

$$wt = buf_c - buf_{min} - ((b_c/b_{min}) * msd) \quad (2)$$

This time is calculated according to Equation 2, where buf_c is the current buffer level, buf_{min} corresponds to the minimum buffer level and b_{min} is the bitrate of the lowest media representation.

B. BOLA

An algorithm that selects the bitrate using only playout buffer occupation information is presented in [13]. BOLA belongs to the buffer-based category and uses an online control algorithm along with the Lyapunov [12] optimization technique to prevent rebuffering events and to increase the quality of the content played by the client.

The quality of the content is represented in BOLA as a utility function based on the average bitrate of the video experienced by the user and the duration of the rebuffering events. BOLA authors claim it can achieve a utility level near the optimum. According to tests in [13], it provides a significantly higher utility than two throughput-based methods ELASTIC [5] and PANDA [10].

IV. RELATIVE SMOOTHED THROUGHPUT

In this section, we propose a new approach to the ST's adaptation algorithm, called the Relative Smoothed Throughput (RST) Method. RST implements a hybrid approach, inheriting components from the throughput-based category as well as from the buffer-based category.

Our approach aims at providing better efficiency for the adaptation process, especially in cases where differences between the representations' bitrates are large.

The RST proposal differs from the original ST algorithm, mainly in the decision criteria used to increase, maintain or reduce the quality of the content of the next segment to be transferred from the server. The following are the changes we propose in relation to the original Smoothed HTTP Throughput method [11].

The decision criteria for increasing the rate is similar to that originally proposed, but the computation of ϵ is performed differently. Instead of using the largest difference between the consecutive representations in which the content is available, we consider the difference between the current representation and the representation one level above. Therefore, we redefine ϵ as ϵ' as indicated in Equation 3. Notice that ϵ' is now variable, according to the current bitrate.

$$\epsilon' = \frac{b_{c+1} - b_c}{b_c} \quad (3)$$

Representation	Bitrate (bps)	ϵ'	ϵ
1	254320	0,994518717	1,001770683
2	507246	0,497888598	1,001770683
3	759798	0,333657104	1,001770683
4	1013310	0,238276539	1,001770683
5	1254758	0,501245659	1,001770683
6	1883700	0,664005946	1,001770683
7	3134488	0,580127919	1,001770683
8	4952892	1,001770683	1,001770683
9	9914554	-	-

Fig. 1. Video coding bitrate / representation example.

In Equation 3, variable b_c represents the bitrate of the current representation, while b_{c+1} corresponds to the bitrate of the representation one unit above. This adjustment allows an improvement in the efficiency of the adaptation technique, as will be demonstrated in Section VI and exemplified below.

Suppose a client using the ST method [11] wishes to consume a content with segments of 4 seconds, encoded in 9 distinct representations, as indicated in Figure 1. Clearly, this client will have difficulty increasing the encoding by one unit up when playing the content in representation 3 on a 1 Mbps link, since in this case ϵ corresponds to approximately 1.00177 and it will rise to level 4 only when μ is greater than 2.00177 ($1 + \epsilon$), which is unlikely, since with the encoding rate of 759798bps of representation 3, the value of μ will be limited by approximately 1.38.

Considering the same example described above, if a player uses the proposed RST method, it will be able to reach representation 4 with the same link of 1 Mbps, because in that case ϵ' will correspond to approximately 0,33366, allowing the client to adapt to reproduce the content in the representation at one unit above, since μ (1.38) will be greater than $1 + \epsilon'$ (1.33366).

In addition to redefining ϵ for better efficiency, the RST proposal contributes to better stability by applying a buffer-based approach, defining three buffer parameters.

The first one is the *safety buffer level* that indicates the minimum buffer size so that a player can raise the level of the representation. This parameter is important to minimize the risk of rebuffering if the network capacity is reduced shortly after the player has requested an increase in the representation.

The last two parameters of the buffer are related to the reduction of the bitrate. The parameter *reduce buffer level* was created in the RST method to improve stability. In the RST method, we reduce the representation to one level below when μ assumes a value less than γ , but only when the buffer is at one level below the parameter *reduce buffer level*.

The last, but not least, parameter is the *minimum buffer level*. It represents the minimum allowed buffer. Its main purpose is to reduce the risk of rebuffering. When the buffer level is below the *minimum buffer level* and μ is less than 1, the bitrate reduction speed is increased, being inversely proportional to the value of μ . In those cases, the player reduces the representation to the largest representation smaller than $\mu * b_c$ where b_c is the current segment bitrate.

As in the ST method, we have adopted a waiting time before requesting the next segment to the server, limiting the amount of buffered media. The difference is that instead of using Equation 2 to calculate the waiting time, we simply use the difference between the current buffer length (in seconds) and the safety buffer level parameter (in seconds). Thus we can achieve a greater acceleration in the initial moments in which the buffer is being filled.

Algorithm 1 presents the logic used by the RST method to define the representation of the next segment to be transferred from the server. It summarizes the adaptation method discussed in this section.

The algorithm checks the current buffer level (line 3) to reduce the bitrate if the buffer is too low. It decreases the next segment representation index to avoid rebuffering if the buffer level is below the minimum buffer level parameter. If μ is greater or equal to one (line 4), the index reduction is of one unit only (line 5). We do this because when the segment fetch time is smaller than the media segment duration, we assume the client will not have much difficulty filling the buffer, so there is no need to reduce the bitrate too much. If μ is less than 1, we do a bitrate reduction proportional to it (line 7), so that the client is able to fill the buffer quickly.

In line 10, RST checks the buffer length again, but this time just to move the representation one unit down if μ is smaller than γ , the switch-down threshold we use to reduce the bitrate based on the smoothed HTTP throughput metric. In order to bring more stability to the algorithm, we only reduce it when the current buffer length is less than the *reduce buffer level* parameter.

Finally, at line 13, if ($\mu > (1 + \epsilon')$) then RST increases the next segment representation by one unit. Here again we check the buffer level before changing the bitrate, in order to reduce instability.

Algorithm 1: Relative Smoothed Throughput

Result: next segment's index and waiting time

```

1  $\mu \leftarrow \text{msd/sft}$ ;
2  $\epsilon' \leftarrow \frac{b_{c+1}-b_c}{b_c}$ ;
3 if  $\text{buf}_c < \text{buf}_{\min}$  then
4   if  $\mu \geq 1$  then
5     return  $\text{index}_c - 1$ ;
6   else
7     return higher index whose bitrate  $< (b_c * \mu)$ ;
8   end
9 end
10 if ( $\mu < \gamma$ ) and ( $\text{buf}_c < \text{buf}_{\text{reduce}}$ ) then
11   return  $\text{index}_c - 1$ ;
12 end
13 if ( $\mu > (1 + \epsilon')$ ) and ( $\text{buf}_c > \text{buf}_{\text{safety}}$ ) then
14   return  $\text{index}_c + 1$ ;
15 end
16 return  $\text{index}_c$ ;

```

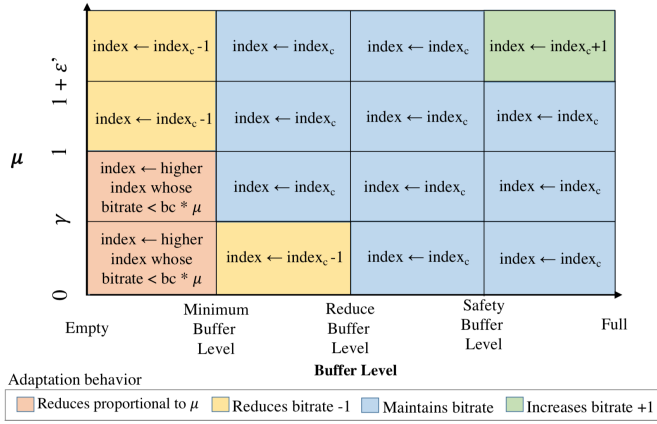


Fig. 2. RST adaptation decision matrix.

Figure 2 shows the behavior of the proposed adaptation algorithm based on the relation between the buffer parameters mentioned previously and the value of μ . This image evidences that our method has a hybrid characteristic, since it considers as much factors related to the transfer rate of the segments as the current level of the buffer.

V. EVALUATION TOOL

In order to evaluate our method, we created a tool called ESFET (Efficiency, Stability and Fairness Evaluation Tool), which can be used together with dash-js [2] player to measure Inefficiency, Instability and Unfairness quality measurements.

In the dash-js client, a scheduler runs the adaptation algorithm every 500ms to check if it must increase, decrease or keep the current representation and then sends the playback data, meaning the chosen representation, bitrate and current timestamp to the ESFET backend.

The backend then adds the client id to the data set and stores this data in a time series database. ESFET has dashboards that read the data from the database and combines them in charts. The graphs shown in Section VI were extracted from ESFET dashboards.

Besides ESFET, we also used dash-js to evaluate our method. Dash-js is an adaptive streaming over HTTP player, available as an open source project at <https://github.com/Dash-Industry-Forum/dash.js>. The version of the dash-js player used as the basis for this work was 2.6.0.

VI. METHOD EVALUATION

The evaluation of the RST method was based on its comparison with two state-of-the-art methods, the Smoothed HTTP Throughput method and the BOLA method, both described in Section III. In order to compare the performance of the three methods we used the Unfairness, Inefficiency and Instability metrics defined in [9] and described in the next subsection.

The Smoothed Streaming Method was included in the evaluation to highlight the advantages of the proposed method over its original.

In order to compare it with a buffer-based adaptation method, we choose BOLA, since this is the buffer-based method chosen by DASH-IF for the dash-js reference client.

A. Evaluation Metrics

In order to describe the quality metrics used in our evaluation, let us consider a scenario where N players are sharing a W bandwidth link and each player x is reproducing content at a $b_{x,t}$ bitrate at time t .

Unfairness is defined by $\sqrt{1 - JainFair}$, where $JainFair$ is the Jain fairness index [8].

JainFair index at a time t is defined as follows:

$$JainFair = \frac{(\sum_{x=1}^N b_{x,t})^2}{N \times \sum_{x=1}^N (b_{x,t})^2} \quad (4)$$

The closer to zero the fairer is the bandwidth sharing.

Inefficiency at time t can be obtained from Equation 5. The closer to zero, the greater the ability of clients running simultaneously to utilize all the bandwidth available, providing better quality in the video consumed by users [6].

$$\frac{\sum_x |b_{x,t} - W|}{W} \quad (5)$$

Instability is another important aspect of user experience. It is measured by Equation 6, which corresponds to a weighted sum of all bitrate changes in the last k seconds, divided by the weighted sum of the bitrates in the last k seconds.

$$\frac{\sum_{d=0}^{k-1} |b_{x,t-d} - b_{x,t-d-1}| * w(d)}{\sum_{d=1}^k b_{x,t-d} \times w(d)} \quad (6)$$

The term $w(d)$ is equal to $k - d$ and it causes a penalty to the most recent bitrate changes.

B. Evaluation Results

During the evaluation, two experiments were performed using these three methods. The first used a player to consume the video content on a 1 Mbps link for 5 minutes. The buffer length during the tests was limited to 30 seconds. The second test used three players consuming the same video simultaneously on a 3 Mbps link for 5 minutes.

We used the Big Buck Bunny [1] video. The segments of the test sample have a duration of 4 seconds. The content used in the test was available in 9 representations, the same shown in Figure 1. In each experiment the following metrics were collected: inefficiency, unfairness and instability, formally defined in [9] as a point of comparison among the methods.

During the first test, there was only one video streaming. This experiment validates the expected behaviour of RST. The results of the first test can be observed in Figures 3 and 4. The first figure shows the representation versus time while the latter presents the three quality measurements: instability, inefficiency and unfairness versus time.

When using the RST method in the first test, the client was able to play the video at an average representation of 4.06 throughout the 5 minutes in which the test was performed.

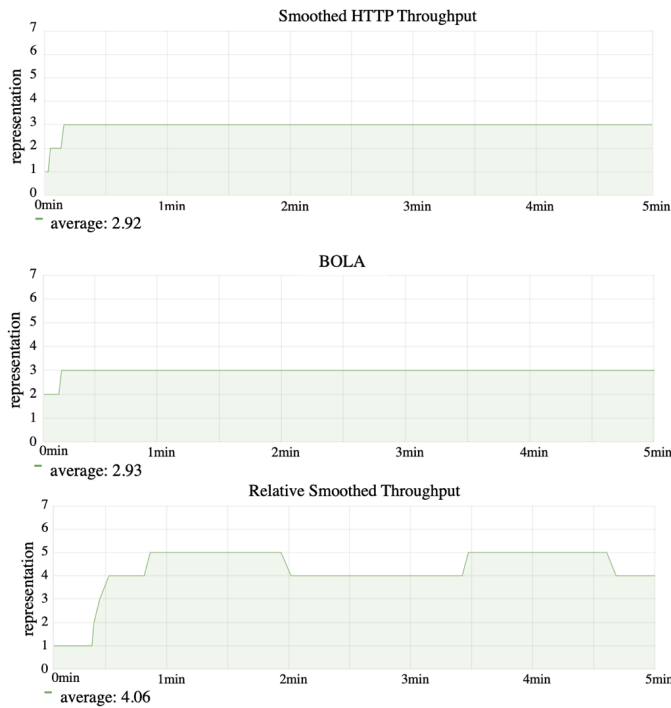


Fig. 3. 1 Player: Representation vs Time.

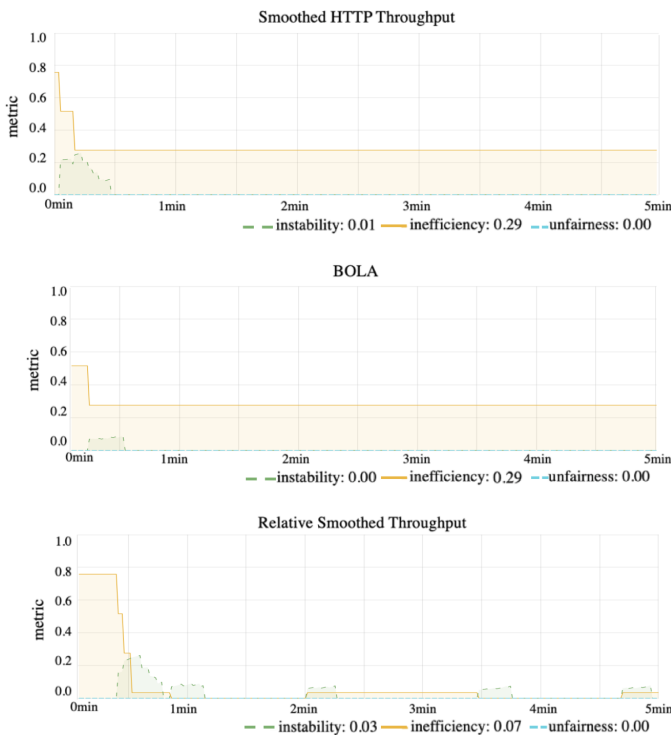


Fig. 4. 1 Player: Quality Measurements vs Time.

Among the three methods evaluated, RST was the only one able to keep the representation average above 4.

During the whole test, both BOLA and the ST method presented an average representation below 3.

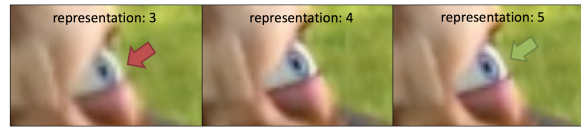


Fig. 5. Differences between representations 3, 4 and 5.

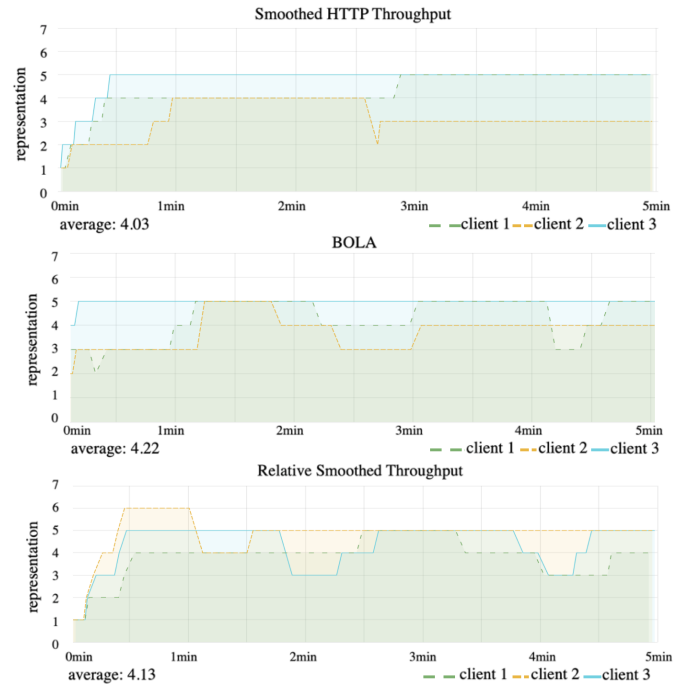


Fig. 6. 3 Players: Representation vs Time.

During the first test, with a single client, RST was the only method capable of achieving representation 5 (1.2Mbps), keeping it for approximately 29% of the test time, while the others did not exceed representation 3 (742Kbps).

In the first test, all three methods presented very similar instability results, as can be seen in figure 4. All of them presented some initial instability. Notice that this behavior is common in adaptive streaming. In general, most adaptation methods start with the lowest representation and go up as the network conditions allow. The Relative Smoothed Throughput Method exhibited 3% instability during the first experiment, but the observed instability was an oscillation between higher quality representations 4 and 5, while the others were fixed in a lower quality representation (representation 3).

The Smoothed HTTP Throughput method and BOLA presented a certain inefficiency during the first experiment. The two methods obtained an inefficiency average of approximately 29%, since both were in representation 3, most of the time. Considering that there was an available bandwidth of 1 Mbps, they could potentially reach representation 4, which did not happen during the first test.

In summary, the Relative Smoothed Throughput method presented a better inefficiency result (0.07) in the efficiency factor, maintaining the rate corresponding to the representation

4 during most of the test time, even reaching representation 5 at some moments, as shown in Figure 3.

Figure 5 shows a visual comparison between representations 3, 4 and 5 obtained by the evaluated methods during the test. It is worth noticing that only RST reached representation 5. The RST method showed a more aggressive behavior than the other two methods evaluated during the test, thus it was more successful in providing a better quality of the contents during the tests.

Table I summarizes the first test results including each metric average and standard deviation. The table shows that ST and BOLA results were more conservative compared to RST in all metrics.

TABLE I
1 PLAYER EXPERIMENTS SUMMARY

	Representation		Inefficiency		Unfairness		Instability	
	Avg.	σ	Avg.	σ	Avg.	σ	Avg.	σ
ST	2.92	0.35	0.29	0.06	0.00	0.00	0.01	0.05
BOLA	2.93	0.26	0.29	0.04	0.00	0.00	0.01	0.02
RST	4.06	1.17	0.07	0.19	0.00	0.00	0.03	0.06

In order to evaluate the proposed method in the fairness dimension, we performed a second test in which three players consumed the same content used in the evaluation with one player, for a period of 5 minutes in which the players shared a 3 Mbps link. The results obtained by the Smoothed HTTP Throughput, BOLA and Relative Smoothed Throughput methods in the second test can be seen in Figure 6.

As can be seen in Figure 6, the Relative Smoothed Throughput again was the only method to reach the highest representation achieved in the test. During the second test, one of its clients achieved representation 6, while all the other clients using other methods were not able to go above representation 5. Figure 6 also shows that both BOLA and RST were able to keep all clients' representation above 3 after the first second.

Table II presents the average and standard deviation of the representations and the three quality measurements: instability, inefficiency and unfairness for the second test.

TABLE II
3 PLAYERS EXPERIMENTS SUMMARY

	Representation		Inefficiency		Unfairness		Instability	
	Avg.	σ	Avg.	σ	Avg.	σ	Avg.	σ
ST	4.03	1.07	0.05	0.14	0.18	0.07	0.03	0.05
BOLA	4.22	0.86	0.03	0.05	0.14	0.08	0.02	0.02
RST	4.13	1.16	0.05	0.15	0.14	0.09	0.04	0.06

In the three-client tests, the RST and BOLA obtained the best unfairness average of 14%, while the Smoothed HTTP Throughput method had the worst unfairness average, 18%. In the second test, BOLA showed the best instability index, with an average of 2%, followed by the Smoothed HTTP Throughput methods with 5% and RST with 6%.

Finally, in the inefficiency dimension, all three methods presented good results with inefficiency below 5%.

VII. CONCLUSIONS

The RST method proposed in this paper consistently provided better efficiency compared to the original ST method. Even though BOLA is considered one of the best methods in the literature, RST presents better efficiency results when running with a single client, and its efficiency results are close to BOLA, when running with multiple clients.

The RST instability metric values seen during the tests were somewhat similar to the ST results. When compared to BOLA, we noticed that there's still room to improve RST's stability, and in future works we intend to address this issue.

ACKNOWLEDGMENT

This work is supported in part by CNPq, CAPES, FAPERJ and CGI/FAPESP.

REFERENCES

- [1] Big buck bunny movie in dash dataset at itec/alpen-adria-universität klagenfurt, 2018. Disponível em <http://www-itec.uni-klu.ac.at/ftp/datasets/DASHDataset2014/BigBuckBunny/>.
- [2] dash.js javascript reference client, 2018. Available in <https://reference.dashif.org/dash.js/>.
- [3] Saamer Akhshabi, Lakshmi Anantakrishnan, Ali C Begen, and Constantine Dovrolis. What happens when http adaptive streaming players compete for bandwidth? In *Proceedings of the 22nd international workshop on Network and Operating System Support for Digital Audio and Video*, pages 9–14. ACM, 2012.
- [4] Saamer Akhshabi, Ali C Begen, and Constantine Dovrolis. An experimental evaluation of rate-adaptation algorithms in adaptive streaming over http. In *Proceedings of the second annual ACM conference on Multimedia systems*, pages 157–168. ACM, 2011.
- [5] Luca De Cicco, Vito Caldaralo, Vittorio Palmisano, and Saverio Mascolo. Elastic: a client-side controller for dynamic adaptive streaming over http (dash). In *2013 20th International Packet Video Workshop*, pages 1–8. IEEE, 2013.
- [6] Florin Dobrian, Vyas Sekar, Asad Awan, Ion Stoica, Dilip Joseph, Aditya Ganjam, Jibin Zhan, and Hui Zhang. Understanding the impact of video quality on user engagement. In *ACM SIGCOMM Computer Communication Review*, volume 41, pages 362–373. ACM, 2011.
- [7] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. A buffer-based approach to rate adaptation: Evidence from a large video streaming service. *ACM SIGCOMM Computer Communication Review*, 44(4):187–198, 2015.
- [8] Raj Jain, Dah-Ming Chiu, and William R Hawe. *A quantitative measure of fairness and discrimination for resource allocation in shared computer system*, volume 38. Eastern Research Laboratory, Digital Equipment Corporation Hudson, MA, 1984.
- [9] Junchen Jiang, Vyas Sekar, and Hui Zhang. Improving fairness, efficiency, and stability in http-based adaptive video streaming with festive. In *Proceedings of the 8th international conference on Emerging networking experiments and technologies*, pages 97–108. ACM, 2012.
- [10] Zhi Li, Xiaoqing Zhu, Josh Gahm, Rong Pan, Hao Hu, Ali C Begen, and Dave Oran. Probe and adapt: Rate adaptation for http video streaming at scale. *arXiv preprint arXiv:1305.0510*, 2013.
- [11] Chenghao Liu, Imed Bouazizi, and Moncef Gabbouj. Rate adaptation for adaptive http streaming. In *Proceedings of the second annual ACM conference on Multimedia systems*, pages 169–174. ACM, 2011.
- [12] Michael J Neely. Stochastic network optimization with application to communication and queueing systems. *Synthesis Lectures on Communication Networks*, 3(1):1–211, 2010.
- [13] Kevin Spiteri, Rahul Ugaonkar, and Ramesh K Sitaraman. Bola: Near-optimal bitrate adaptation for online videos. In *INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*, IEEE, pages 1–9. IEEE, 2016.
- [14] Thomas Stockhammer. Dynamic adaptive streaming over http: standards and design principles. In *Proceedings of the second annual ACM conference on Multimedia systems*, pages 133–144. ACM, 2011.
- [15] Dinesh C Verma. *Content distribution networks: an engineering approach*. John Wiley & Sons, 2003.